New issue

Exercise: Getting Started with GitHub Copilot #1

Closed

github-actions bot opened 51m ago Contributor

Getting Started with GitHub Copilot

👋 Hey there @pramodoutlook! Welcome to your Skills exercise!

Welcome to the exciting world of GitHub Copilot! 🚀 In this exercise, you'll unlock the potential of this AI-powered coding assistant to accelerate your development process. Let's dive in and have some fun exploring the future of coding together! 💻✨

✨ **This is an interactive, hands-on GitHub Skills exercise!**

As you complete each step, I'll leave updates in the comments:

- ✅ Check your work and guide you forward
- 💡 Share helpful tips and resources
- 🚀 Celebrate your progress and completion

Let's get started - good luck and have fun!

— Mona

> If you encounter any issues along the way please report them here.

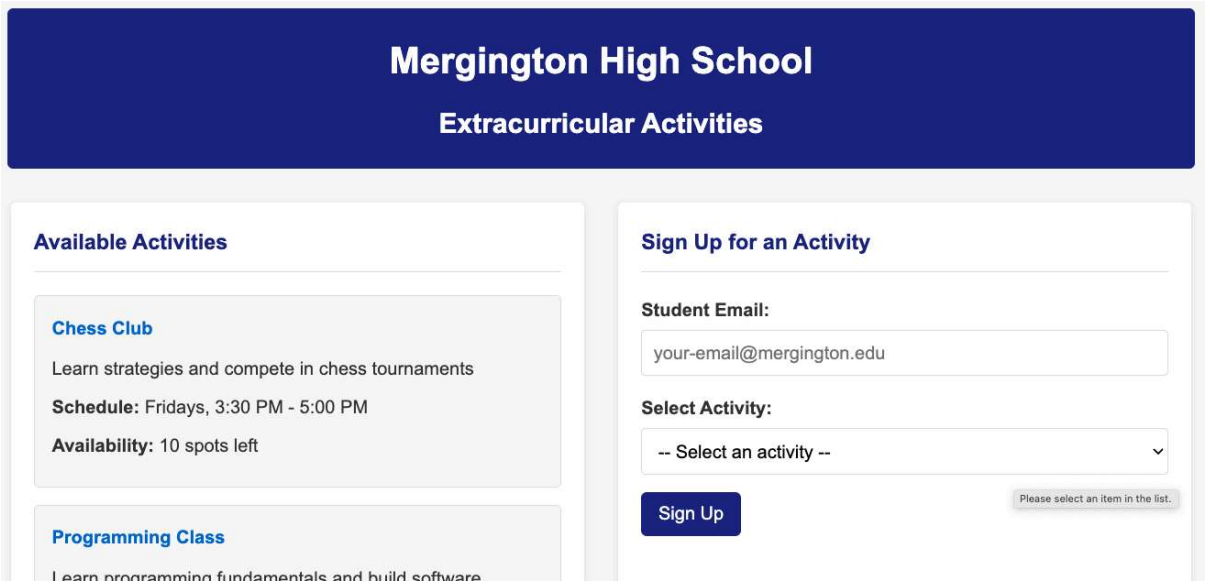
Create sub-issue

github-actions bot 51m ago – with GitHub Actions Contributor Author

Step 1: Hello Copilot

Welcome to your "**Getting Started with GitHub Copilot**" exercise! 🤖

In this exercise, you will be using different GitHub Copilot features to work on a website that allows students of Mergington High School to sign up for extracurricular activities. 🎻 ⚽ ♟️



📖 Theory: Getting to know GitHub Copilot

GitHub Copilot is an AI coding assistant that helps you write code faster and with less effort, allowing you to focus more energy on problem solving and collaboration.

GitHub Copilot has been proven to increase developer productivity and accelerate the pace of software development. For more information, see Research: quantifying GitHub Copilot's impact on developer productivity and happiness in the GitHub blog.

As you work in your IDE, you'll most often interact with GitHub Copilot in the following ways:

Interaction Mode	📝 Description	🎯 Best For
⚡ Inline suggestions	AI-powered code suggestions that appear as you type, offering context-aware completions from single lines to entire functions.	Completion of the current line, sometimes a whole new block of code
💭 Inline Chat	Interactive chat scoped to your current file or selection. Ask questions about specific code blocks.	Code explanations, debugging specific functions, targeted improvements
💬 Ask Mode	Optimized for answering questions about your codebase, coding, and general technology	Understanding how code works, brainstorming ideas, asking

Interaction Mode	📝 Description	🎯 Best For
	concepts.	questions
Agent Mode	Recommended default mode for most coding tasks: autonomous edits, tool use, and follow-through until the task is done.	Daily coding tasks, from scoped fixes to larger multi-file implementation work
Plan Agent	Optimized for drafting a plan and asking clarifying questions before any code changes are made.	When you want a reviewed plan first, then hand off to implementation

As you work, you'll find GitHub Copilot can help out in several places across the `github.com` website and in your favorite coding environments such as VS Code, Jet Brains, and Xcode!

For today's coding though, we will practice with VS Code in a pre-configured development environment known as a GitHub Codespace.

> **Tip**
>
> You can learn more about current and upcoming features in the GitHub Copilot Features documentation.

⌨️ Activity: Get a project intro from Copilot Chat

Let's start up our development environment, use copilot to learn a bit about the project, and then give it a test run.

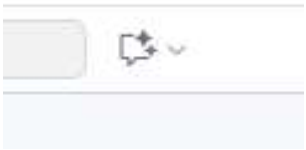
1. Use the below button to open the **Create Codespace** page in a new tab. Use the default configuration.

   Open in GitHub Codespaces

2. Confirm the **Repository** field is your copy of the exercise, not the original, then click the green **Create Codespace** button.
   - ✅ Your copy: `/pramodoutlook/skills-getting-started-with-github-copilot`
   - ❌ Original: `/skills/getting-started-with-github-copilot`
3. Wait a moment for Visual Studio Code to load in your browser.
4. In the left sidebar, click the extensions tab and verify that the `GitHub Copilot Chat` and `Python` extensions are installed and enabled.

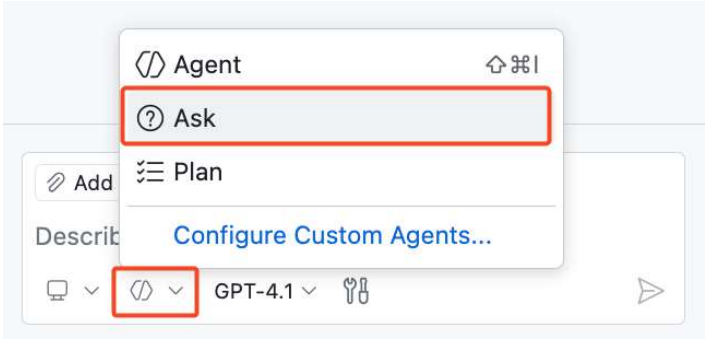
▶ 🔎 GitHub Copilot Chat extension missing ❓

5. At the top of VS Code, locate and click the **Toggle Chat icon** to open a Copilot Chat side panel.



> 💡 **Note:** If this is your first time using GitHub Copilot, you may need to accept the usage terms to continue.

6. Make sure you are in **Ask Mode** for our first interaction

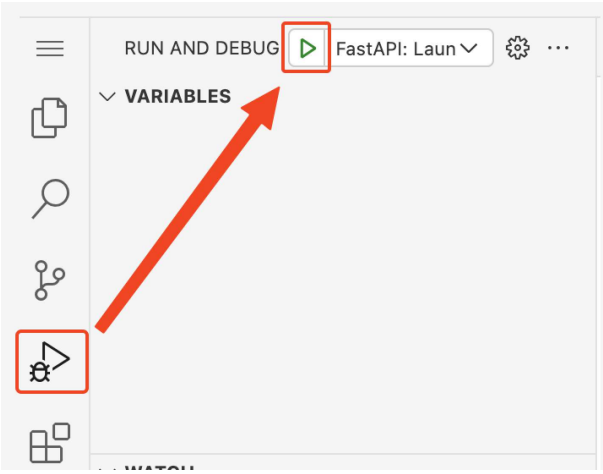


7. Enter the below prompt to ask Copilot to introduce you to the project.

> **Prompt**
>
> ```
> Please briefly explain the structure of this project.
> What should I do to run it?
> ```

> 💡 **Note:** It is not necessary to follow Copilot's recommended instructions. We have already prepared the environment for you.

8. Now that we know a bit more about the project, let's actually try running it! In the left sidebar, select the `Run and Debug` tab and then press the **Start Debugging** icon.



9. We want to see our webpage running in a browser, so let's find the url and port. If it isn't visible, expand the lower panel and select the **Ports** tab.

10. In the list, find port `8000` and the related link. Hover over the link and select the **Open in browser** icon.

⌨️ Activity: Use Copilot to help remember a terminal command 🤦

Great work! Now that we are familiar with the app and we know it works, let's ask copilot for help starting a branch so we can do some customizing.

1. In VS Code's bottom panel, select the **Terminal** tab and on the right side click the plus `+` sign to create a new terminal window.

> 🪧 **Note:** This will avoid stopping the existing debug session that is hosting our web application service.

2. Within the new terminal window use the keyboard shortcut `Ctrl + I` (windows) or `Cmd + I` (mac) to bring up **Copilot's Terminal Inline Chat**.

3. Let's ask Copilot to help us remember a command we have forgotten: creating a branch and publishing it.

> **Prompt**
>
> ```
> Hey copilot, how can I create and publish a new Git branch called "accelerate-with-copilot"?
> ```

> 💡 **Tip:** If Copilot doesn't give you quite what you want, you can always continue explaining what you need. Copilot will remember the conversation history for follow-up responses.

4. Press the `Run` button to let Copilot insert the terminal command for us. No need to copy and paste!

5. After a moment, look in the VS Code lower status bar, on the left, to see the active branch. It should now say `accelerate-with-copilot`. If so, you are all done with this step!

6. Now that your branch is pushed to GitHub, Mona should already be busy checking your work. Give her a moment and keep watch in the comments. You will see her respond with progress info and the next lesson.

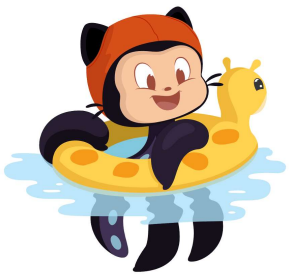
▶ Having trouble? 🤷

github-actions bot
51m ago – with GitHub Actions

Last edited by github-actions · Contributor · Author · •••

Step 1 - Passed ✅

Status	Description
✅ - Pass	Checked that branch name is accelerate-with-copilot



github-actions bot 34m ago – with GitHub Actions

Contributor · Author · •••

🎉 🎉 🎉 Nice work! Everything is perfect! 🎉 🎉 🎉
Preparing content for step 2! One moment... 🤓

github-actions bot 34m ago – with GitHub Actions

Contributor · Author · •••

Step 2: Getting work done with Copilot

In the previous step, GitHub Copilot was able to help us onboard to the project. That alone is a huge time saver, but now let's get some work done!

🐛 THERE IS A BUG ON THE WEBSITE 🐛

We've discovered that something's off in the signup flow.
Students can currently register for the same activity **more than once**! Let's see how far Copilot can take us in uncovering the cause and shaping a clean fix.

Before we dive in, a quick primer on how Copilot works. 👩‍🏫

📖 Theory: How Copilot works

In short, you can think of Copilot like a very specialized coworker. To be effective with them, you need to provide them background (context) and clear direction (prompts). Additionally, different people are better at different things because of their unique experiences (models).

- **How do we provide context?:** In our coding environment, Copilot will automatically consider nearby code and open tabs. If you are using chat, you can also explicitly refer to files.
- **What model should we pick?:** For our exercise, it shouldn't matter too much. Experimenting with different models is part of the fun! That's another lesson! 🤖
- **How do I make prompts?:** Being explicit and clear helps Copilot do the best job. But unlike some traditional systems, you can always clarify your direction with followup prompts.

> **Tip**
>
> There several other ways to supplement Copilot's knowledge and capabilities like chat participants, chat variables, slash commands, and MCP tools.

⌨️ Activity: Use Copilot to fix our registration bug 🐛

1. Let's ask Copilot to suggest where our bug might be coming from. Open the **Copilot Chat** panel in **Ask mode** and ask the following.

   > **Prompt**
   >
   > ```
   > #codebase Students are able to register twice for an activity.
   > Where could this bug be coming from?
   > ```

2. Now that we know the issue is in the `src/app.py` file and the `signup_for_activity` method, let's follow Copilot's recommendation and go fix it (semi-manually). We'll start with a comment and let Copilot finish the correction.

i. Open the `src/app.py` file.

> 💡 **Tip:** If Copilot mentioned `src/app.py` in chat, you can click the file directly in the chat view to open it.

ii. Near the bottom of the file, find the `signup_for_activity` function.

iii. Find the comment line that describes adding a student. Above this is where it seems logical to do our registration check.

iv. Enter the below comment and press enter to go to the next line. After a moment, temporary shadow text will appear with a suggestion from Copilot! Nice! 🎉

Comment:

```
# Validate student is not already signed up
```

v. Press `Tab` to accept Copilot's suggestion and convert the shadow text to code.

▶ Example Results

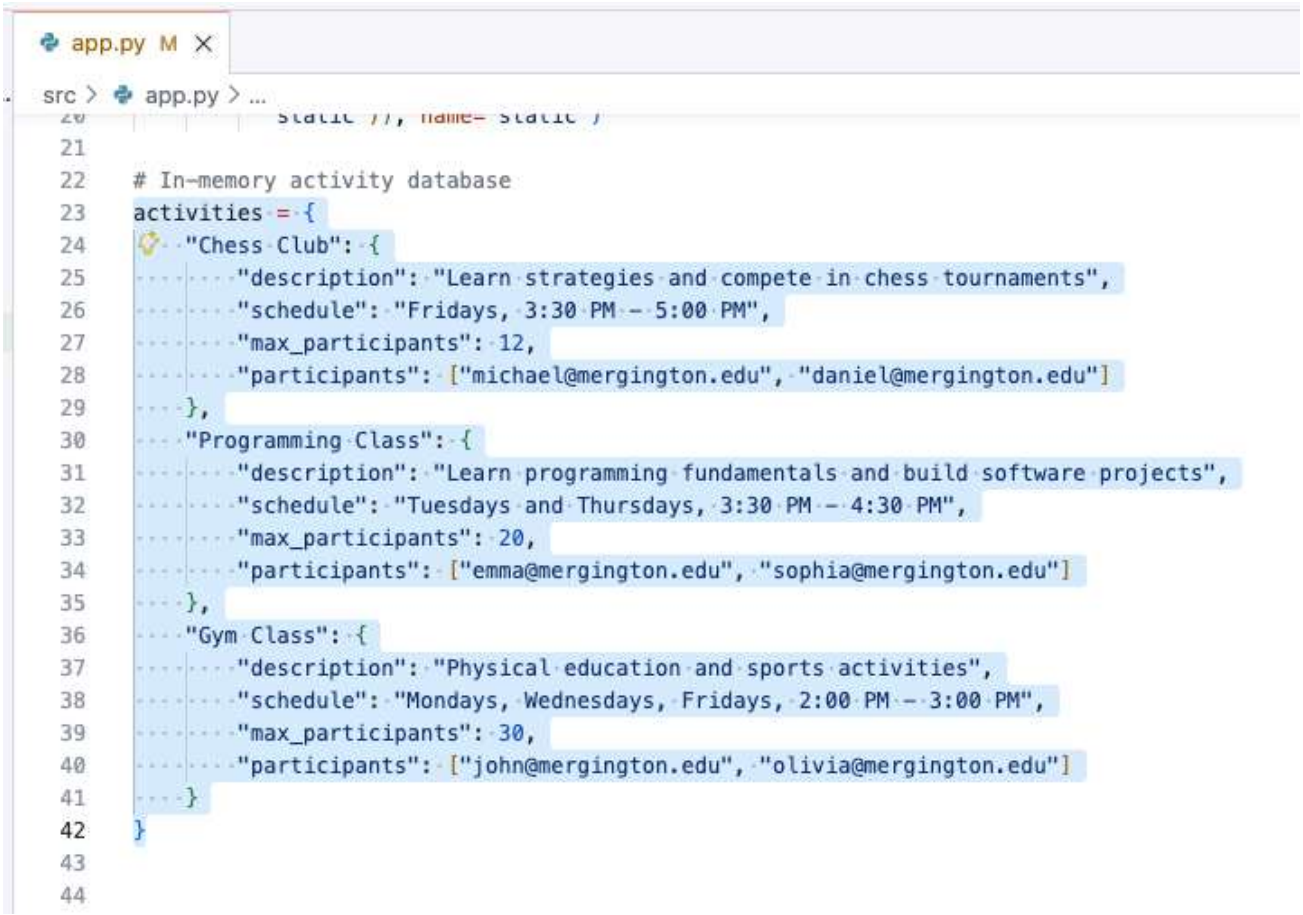
⌨️ Activity: Let Copilot generate sample data 📋

In new project developments, it's often helpful to have some realistic looking fake data for testing. Copilot is excellent at this task, so let's add some more sample activities and introduce another way to interact with Copilot using **Inline Chat**

Inline Chat and the **Copilot Chat** panel are similar, but differ in scope: Copilot Chat handles broader, multi-file or exploratory questions; Inline Chat is faster when you want targeted help on the exact line or block in front of you.

1. Near the top of the `src/app.py` file (about line 23), find the `activities` variable, where our example extracurricular activities are configured.

2. Highlight the entire `activities` dictionary by clicking and dragging your mouse from the top to the bottom of the dictionary. This will help provide context to Copilot for our next prompt.



3. Bring up Copilot inline chat by using the keyboard command `Ctrl + I` (windows) or `Cmd + I` (mac).

> 💡 **Tip:** Another way to bring up Copilot inline chat is: `right click` on any of the selected lines -> `Open Inline Chat`.

4. Enter the following prompt text and press enter or the **Send** button on the right.

> **Prompt**
>
> ```
> Add 2 more sports related activities, 2 more artistic
> activities, and 2 more intellectual activities.
> ```

5. After a moment, Copilot will directly start making changes to the code. The changes will be stylized differently to make any additions and removals easy to identify. Take a moment to inspect and verify the changes, and then press the **Keep** button.

   ▶ Example Results

6. You can now go to your website and verify that the new activities are visible.

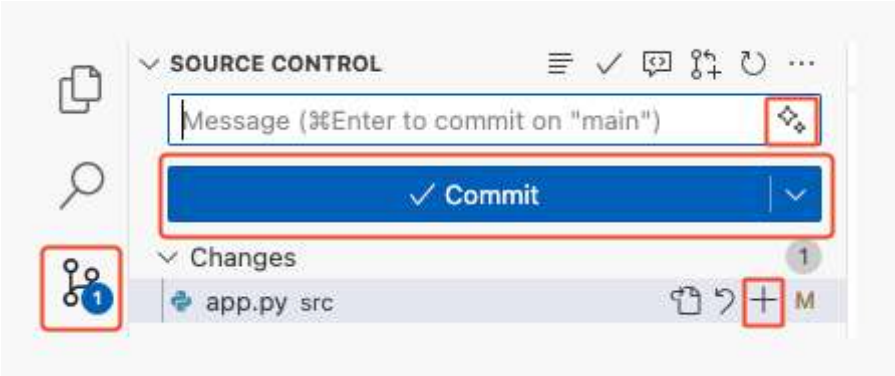
⌨️ Activity: Use Copilot to describe our work 💬

Nice work fixing that bug and expanding the example activities! Now let's get our work committed and pushed to GitHub, again with the help of Copilot!

1. In the left sidebar, select the `Source Control` tab.

   > 💡 **Tip:** Opening a file from the source control area will show the differences to the original rather than simply opening it.

2. Find the `app.py` file and press the `+` sign to collect your changes together in the staging area.



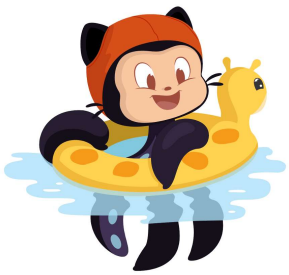
3. Above the list of staged changes, find the **Message** text box, but **don't enter anything** for now.
   - Typically, you would write a short description of the changes here, but now we have Copilot to help out!
4. To the right of the **Message** text box, find and click the **Generate Commit Message** button (sparkles icon).
5. Press the **Commit** button and **Sync Changes** button to push your changes to GitHub.
6. Wait a moment for Mona to check your work, provide feedback, and share the next lesson.

▶ Having trouble? 🤷

github-actions bot
34m ago – with GitHub Actions

Last edited by github-actions ▾ Contributor Author •••

Step 2 - Passed ✅



Status	Description
✅ - Pass	New activities added to src/app.py. We found 9 activities (minimum 4 required)

github-actions bot 25m ago – with GitHub Actions Contributor Author ···

🎉 🎉 🎉 Nice work! Everything is perfect! 🎉 🎉 🎉
Preparing content for step 3! One moment... 🤓

github-actions bot 25m ago – with GitHub Actions Contributor Author ···

Step 3: Engage Hyperdrive - Copilot Agent Mode 🚀

📖 Theory: What is Copilot Agent Mode?

Copilot agent mode is the next evolution in AI-assisted coding. Acting as an autonomous peer programmer, it performs multi-step coding tasks at your command.

Copilot Agent Mode responds to compile and lint errors, monitors terminal and test output, and auto-corrects in a loop until the task is completed.

Agent Mode (at a glance)

Aspect	👩‍🚀 Agent Mode
Autonomy and planning	Breaks down high-level requests into multi-step work and iterates until the task is complete.
Context gathering	Uses your current context and can discover additional relevant files when needed.
Tool use	Selects and invokes tools automatically; you can also direct tools with mentions like `#codebase`.
Approval and safety gates	Sensitive actions can require approval before execution, helping you stay in control.

🧰 Agent Mode Tools

Agent mode uses tools to accomplish specialized tasks while processing a user request. Examples of such tasks are:

- Finding relevant files to complete your prompt
- Fetching contents of a webpage
- Running tests or terminal commands

> **Tip**
>
> While VS Code provides many built-in tools, you can also provide Agent Mode more domain-specific powers through **MCP tools**.
>
> Read more on MCP servers and GitHub MCP Server

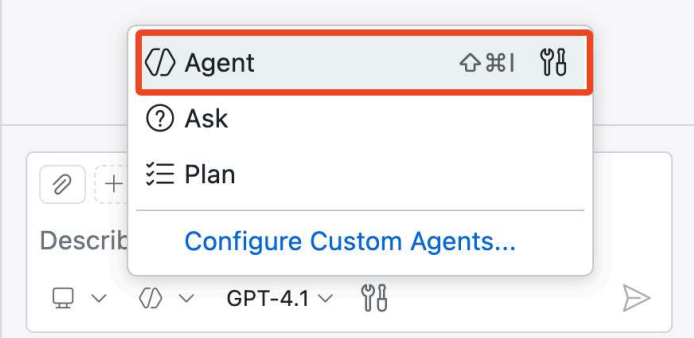
Now, let's give **Agent Mode** a try! 👩‍🚀

⌨️ Activity: Use Copilot to add a new feature! 🚀

Our website lists activities, but it's keeping the guest list secret 🤫

Let's use Copilot to change the website to display signed up students under each activity!

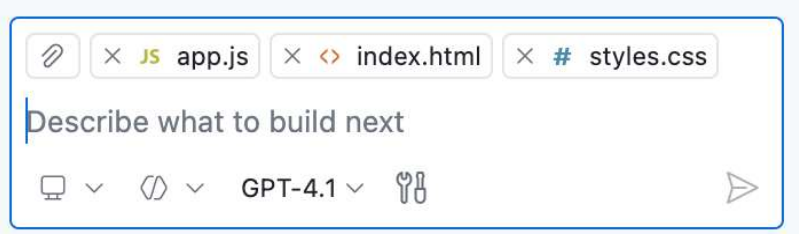
1. At the bottom of Copilot Chat window, use the dropdown to switch to **Agent** mode.



2. Open the files related to our webpage then drag each editor window (or file) to the chat panel, informing Copilot to use them as context.

   - `src/static/app.js`
   - `src/static/index.html`
   - `src/static/styles.css`

   > 🪧 **Note:** Adding files as context is optional. If you skip this, Copilot Agent Mode can still use tools like `#codebase` to search for relevant files from your prompt. Adding specific files helps point Copilot in the right direction, which is especially useful in larger codebases.



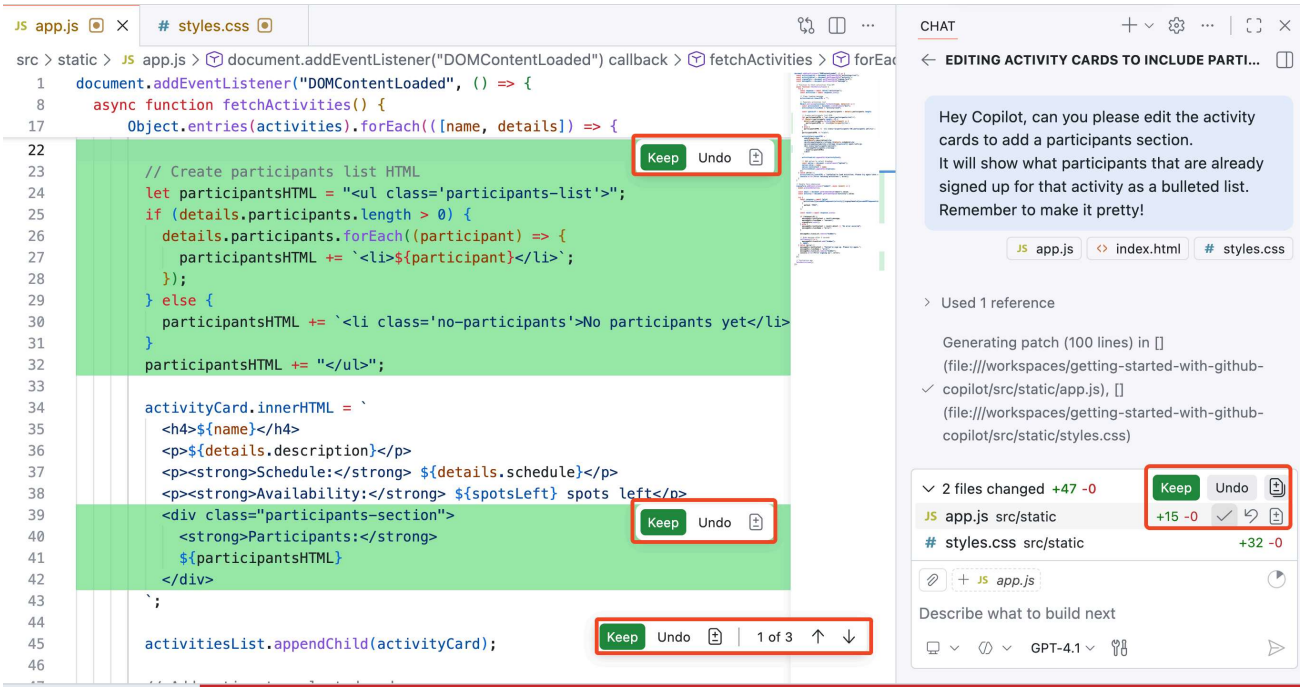
   > 💡 **Tip:** You can also use the **Add Context...** button to provide other sources of context items, like a GitHub issue or the results of a terminal window.

3. Ask Copilot to update our project to display the current participants of activities. Wait a moment for the edit suggestions to arrive and be applied.

   > **Prompt**
   >
   > ```
   > Hey Copilot, can you please edit the activity cards to add a participants section.
   > It will show what participants that are already signed up for that activity as a bulleted
   > list.
   > Remember to make it pretty!
   > ```

   After Copilot finishes work, you are in control of what changes get to stay.

   Using the **Keep** buttons shown below, you can accept/discard all changes or review and decide change by change. This can be done either from the chat panel view or while inspecting each edited file.



4. Before we simply accept the changes, please check our website again and verify everything is updated as expected.

   Here is an example of an updated activity card. You may need to restart the app or refresh the page.

> 🪧 **Note:** Your activity card may look different. Copilot won't always produce the same results.

▶ Need help? 🤷

5. Now that we have confirmed our changes are good, use the panel to cycle through each suggested edit and press **Keep** to apply the change.

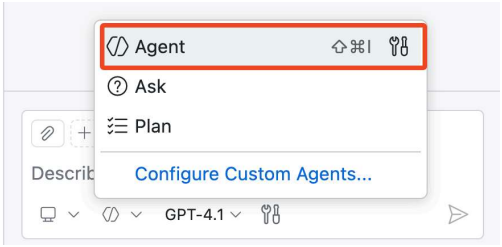
> 💡 **Tip:** You can accept the changes directly, modify them, or provide additional instruction to refine them using the chat interface.

⌨️ Activity: Use Agent mode to add functional "unregister" buttons

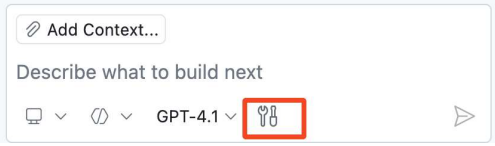
Let's experiment with some more open-ended requests that will add more functionality to our web application.

If you don't get the desired results, you can try other models or provide follow-up feedback to refine the results.

1. Make sure your Copilot is still in **Agent** mode.



2. Click on the **Tools** icon and explore all Tools currently available to Copilot Agent Mode.



3. Time for our test! Let's ask Copilot to add functionality for removing participants.

> **Prompt**
>
> ```
> #codebase Please add a delete icon next to each participant and hide the bullet points
> When clicked, it will unregister that participant from the activity.
> ```

The `#codebase` tool is used by Copilot to find relevant files, code chunks that are relevant to the task at hand.

> 🪧 **Note:** In this lab we explicitly include the `#codebase` tool to get the most repeatable results. Feel free to try the prompt **without** `#codebase` and observe whether Agent Mode decides to gather broader project context on its own.

4. When Copilot is finished, inspect the code changes and the results on the website. If you like the results, press the **Keep** button. If not, try providing Copilot some feedback to refine the results.

> 🪧 **Note:** If you don't see updates on the website, you may need to restart the debugger

5. Ask Copilot to fix a registration bug.

> 💡 **Tip:** We recommend testing the registration flow yourself so you can clearly see the before/after changes behavior.

> **Prompt**
>
> ```
> I've noticed there seems to be a bug.
> When a participant is registered, the page must be refreshed to see the change on the
> ```

```
activity.
```

6. When Copilot is finished, inspect the results and validate the registration flow on the website.

   If you like the results, press the **Keep** button. If not, try providing Copilot some feedback.

7. **Commit** and **push** all your changes to the `accelerate-with-copilot` branch.

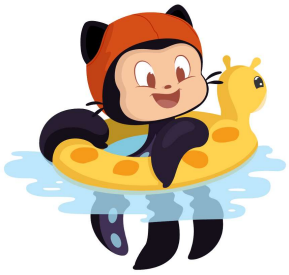
8. Wait for Mona to check your work and share the next step.

github-actions bot
25m ago – with GitHub Actions

Last edited by github-actions · Contributor · Author · •••

Step 3 - Passed ✅

Status	Description
✅ - Pass	Checked app.js for participant info
✅ - Pass	Checked styles.css for participant info



github-actions bot 10m ago – with GitHub Actions

Contributor · Author · •••

🎉 🎉 🎉 Nice work! Everything is perfect! 🎉 🎉 🎉
Preparing content for step 4! One moment... 🤓

github-actions bot 10m ago – with GitHub Actions

Contributor · Author · •••

Step 4: Plan your implementation with the Planning Agent 🧭

In the last step, Agent Mode helped us move fast and ship new functionality. 🚀

Now let's slow down for one round and work like architects: define a strong testing approach first, then hand it off for implementation. This gives us better clarity, fewer surprises, and cleaner results. 🧪

📖 Theory: What is Copilot Plan Agent?

Copilot Plan Agent helps you design a solution before any code is changed.

Instead of jumping straight into edits, it researches your request, asks clarifying questions, and drafts an implementation plan you can refine.

Plan Agent (at a glance)

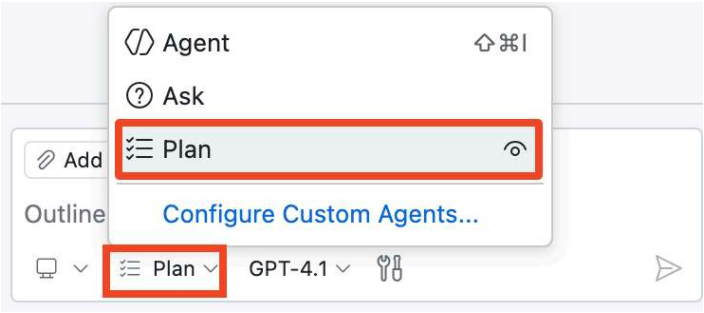
Aspect	🧭 Plan Agent
Purpose	Creates a structured implementation plan before coding starts.
Context gathering	Uses read-only research to understand requirements and constraints.
Collaboration style	Asks clarifying questions, then updates the plan using your answers.
Iteration	Supports multiple refinement passes before implementation.
Safety	Does not edit files until you approve the plan and hand off to **Agent Mode**.
Handoff	**Start implementation** button hands off the approved plan to **Agent Mode** for coding.

> **Tip**
>
> You can start from a high-level request and then add constraints and details in follow-up prompts.

⌨️ Activity: Plan and implement backend tests

Your backend still has zero test coverage. Use **Plan Agent** to create a plan, answer questions, and then launch implementation.

1. Open the **Copilot Chat** panel and switch to **Plan Agent**.



2. Let's start with a broad prompt and Copilot will help us fill in the details:

> **Prompt**
>
> ```
> Let's plan for adding backend FastAPI tests in a separate tests directory.
> ```

3. Wait for Copilot to generate its first plan. If it asks you any questions, answer them to the best of your ability.

> **Note:** Don't worry about getting it perfect, you can always refine the plan later.

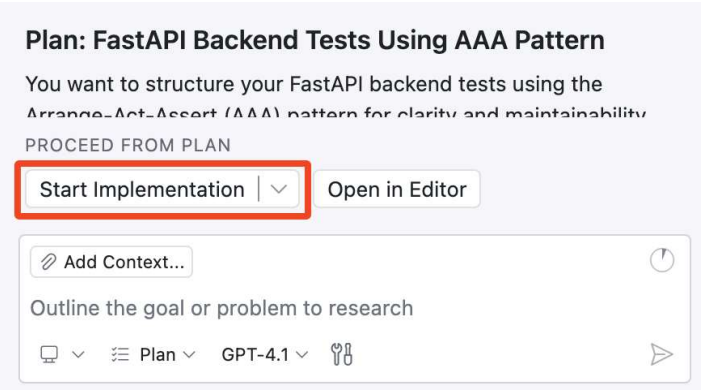
4. You can refine the plan and provide additional details in follow up prompts

   Here are some examples:

> **Prompt**
>
> ```
> Let's use the AAA (Arrange-Act-Assert) testing pattern to structure our tests
> ```

> **Prompt**
>
> ```
> Make sure we use `pytest` and add it to `requirements.txt` file
> ```

5. Review the proposed plan and when you are happy with it, click **Start implementation** to hand off to **Agent Mode**.



Notice that clicking the button switched from **Plan** to **Agent Mode**. From this point on, Copilot can edit your codebase, just like before.

6. Watch Copilot implement the plan you just created. It may ask for permissions to run certain tools (e.g., run commands or create virtual environments). Approve these permissions so it can continue working.

7. Review the changes and make sure tests run successfully. If needed, continue guiding until implementation is complete.

   🎯 **Goal: Get all tests passing (green) before you move on.** ✅

   > 🪧 **Note:** Agent Mode may complete this in one pass, or it may need follow-up prompts from you.

8. **Commit** and **push** all your changes to the `accelerate-with-copilot` branch.

9. Wait for Mona to check your work and share the next step.

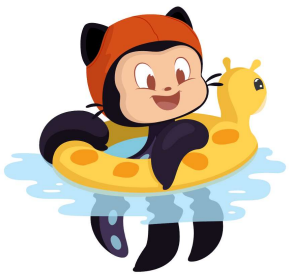
▶ Having trouble? 🤷

github-actions bot
10m ago – with GitHub Actions

Last edited by github-actions · Contributor · Author · •••

Step 4 - Passed ✅

Status	Description
✅ - Pass	Checked for pytest in requirements.txt
✅ - Pass	Checked if tests are added in a separate directory



github-actions bot 5m ago – with GitHub Actions Contributor Author

🎉 🎉 🎉 Nice work! Everything is perfect! 🎉 🎉 🎉
Preparing content for step 5! One moment... 🤓

github-actions bot 5m ago – with GitHub Actions Contributor Author

Step 5: Using GitHub Copilot within a pull request

Congratulations! You are finished with coding for this exercise (and VS Code). Now it's time to merge our work. 🎉 To wrap up, let's learn about two limited-access Copilot features that can speed up our pull requests!

📖 Theory: GitHub Copilot for pull requests

Copilot pull request summaries

Typically, you would review your notes and commit messages then summarize them for your pull request description. This may take some time, especially if commit messages are inconsistent or code is not documented well. Fortunately, Copilot can consider all changes in the pull request and provide the important highlights, and with references too!

Copilot code review

More eyes on our work is always useful so let's ask Copilot to do a first pass before we do a normal peer review process. Copilot is great at catching common mistakes that are fixed by simple adjustments, but please remember to use it responsibly.

> **Note**
>
> These features are only available on paid plans of **GitHub Copilot**. [docs]

⌨️ Activity: Summarize and review a PR with Copilot

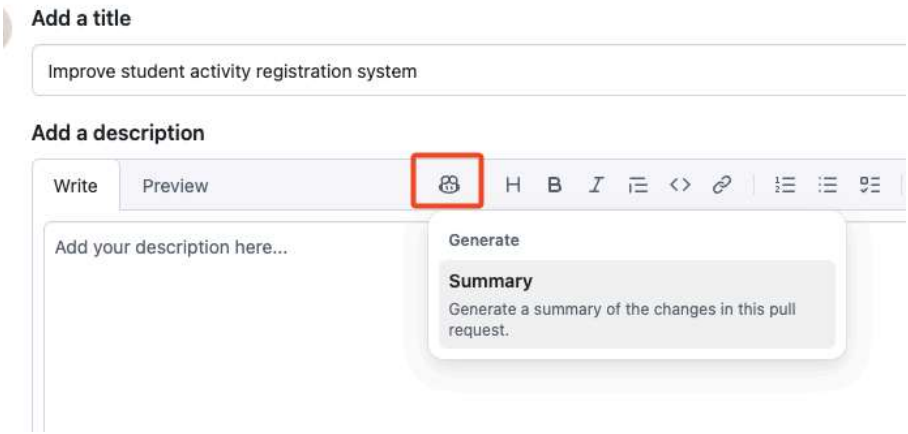
Both **Copilot pull request summaries** and **Copilot code review** have limited access, so this activity is mostly optional. If you don't have access, skip the optional steps of this activity.

1. In a web browser, open another tab and navigate to your exercise repository.

2. You might notice a **notification banner** suggesting to create a new pull request. Click that or use the **Pull Requests** tab at the top to **create a new pull request**. Please use the following details:

   - **base:** `main`
   - **compare:** `accelerate-with-copilot`
   - **title:** `Improve student activity registration system`

3. (Optional) In the PR description toolbar click the **Copilot** icon and **Summary** action. After a moment, Copilot will add a description based on your changes. 📝



4. (Optional) In the right side information panel at the top, locate the **Reviewers** section and click the **Request** button next to a **Copilot icon**. Wait a moment for Copilot to add a review comment to your pull request!

> 💡 **Tip:** Notice a log entry that Copilot was requested for a review.

5. At the bottom, press the **Merge pull request** button. Nice work! You are all done! 🎉

6. Wait a moment for Mona to check your work, provide feedback, and post a final review of this exercise!

github-actions bot 4m ago – with GitHub Actions Contributor Author

Please, follow the steps above.
I'll watch your progress in the background to provide feedback. 🧐

github-actions bot 2m ago – with GitHub Actions Contributor Author •••

🎉 🎉 🎉 Nice work! Everything is perfect! 🎉 🎉 🎉
Now, let's do a quick review!

github-actions bot 2m ago – with GitHub Actions Contributor Author •••

Review

Congratulations, you've completed this exercise and learned a lot about GitHub Copilot!

Here's a recap of the GitHub Copilot features you learned:

- **Ask Mode**: Explored your codebase with Copilot
- **Inline suggestions**: Completed code with Tab acceptance
- **Inline Chat**: Generated code and data with Ctrl/Cmd + I
- **Agent Mode**: Built features autonomously
- **Plan Agent**: Drafted a plan, answered questions, and started implementation
- **GitHub integration**: Generated commit messages, PR summaries, and code reviews

What's next?

- Check out the other GitHub Skills exercises.
  - Learn how to Integrate MCP with Copilot to give Copilot extra capabilities!
  - Tailor Copilot to your project needs in Customize your GitHub Copilot Experience
  - Tackle legacy COBOL code in Modernize Your Legacy Code with GitHub Copilot exercise
  - Try GitHub Copilot Coding Agent in the Expand your team with Copilot exercise

github-actions bot 2m ago – with GitHub Actions Contributor Author •••

Congratulations **@pramodoutlook**! You finished the exercise! 🎉 🎉 🎉

We've updated the repository with a couple changes to highlight your success!

Return to the repository home page to see your progress!

github-actions closed this as completed 2m ago

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Reopen issue Comment

Metadata

Assignees

No one - Assign yourself

Labels

No labels

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Create a branch for this issue or link a pull request.

Notifications Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

No participants

Open in GitHub Copilot app

Transfer issue

Clone issue

Lock conversation

Pin issue

Delete issue

Give feedback